

Introducción a ORM

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

-  Entender qué es un ORM y por qué usarlo
-  Conocer las ventajas y desventajas de los ORMs
-  Entender por qué elegimos SQLAlchemy
-  Diferenciar entre SQLAlchemy Core y ORM

Contenido

 Concepto de ORM

1. ¿Qué es un ORM?

ORM (Object-Relational Mapping) es una técnica que permite interactuar con bases de datos relacionales usando objetos de programación en lugar de SQL directo.

```
# ❌ Sin ORM - SQL directo (propenso a errores, SQL injection)
cursor.execute("""
    INSERT INTO users (name, email, age)
    VALUES (?, ?, ?)
""", ("John", "john@example.com", 25))

# ✅ Con ORM - Objetos Python (seguro, tipado, mantenible)
user = User(name="John", email="john@example.com", age=25)
session.add(user)
session.commit()
```

2. El Mapeo Objeto-Relacional

Concepto SQL	Concepto ORM
Tabla	Clase (Model)
Columna	Atributo
Fila	Instancia (objeto)
Foreign Key	Relación
Query	Método/Expresión

```
# La tabla "users" se convierte en la clase User
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True)
    age: Mapped[int | None] = mapped_column(default=None)
```

```
# Una fila se convierte en un objeto
user = User(name="John", email="john@example.com")
print(user.name) # "John" - acceso como atributo
```

3. Ventajas de Usar un ORM

✅ Seguridad

```
# ❌ SQL Injection vulnerable
query = f"SELECT * FROM users WHERE email = '{user_input}'"
# Si user_input = "'; DROP TABLE users; --" → Desastre!

# ✅ ORM previene SQL injection automáticamente
stmt = select(User).where(User.email == user_input)
# Los parámetros siempre se escapan correctamente
```

✅ Productividad

```
# Código más corto y legible
user = User(name="John", email="john@example.com")
session.add(user)
session.commit()

# vs escribir INSERT statements manualmente
```

✅ Portabilidad

```
# El mismo código funciona con diferentes bases de datos
# Solo cambia la URL de conexión

# SQLite (desarrollo)
SQLALCHEMY_DATABASE_URL = "sqlite:///./app.db"

# PostgreSQL (producción)
SQLALCHEMY_DATABASE_URL = "postgresql://user:pass@localhost/db"
```

✅ Mantenibilidad

```
# Modelos centralizados, fáciles de modificar
class User(Base):
    __tablename__ = "users"

    # Agregar un campo es simple
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

4. Desventajas de los ORMs

⚠️ Curva de Aprendizaje

```
# Necesitas aprender la API del ORM
# SQLAlchemy tiene muchos conceptos:
```

```
# - Engine, Session, Connection
# - Mapped, mapped_column
# - select, insert, update, delete
# - Relationships, lazy loading, eager loading
```

⚠ Abstracción con Costo

```
# Queries complejas pueden ser menos eficientes
# El ORM genera SQL que quizás no es óptimo

# A veces necesitas SQL raw para optimización
stmt = text("SELECT * FROM users WHERE ...")
result = session.execute(stmt)
```

⚠ Debugging Más Difícil

```
# Errores pueden venir del ORM, no de tu código
# Necesitas entender qué SQL se genera

# Tip: Habilitar logging de SQL
import logging
logging.getLogger('sqlalchemy.engine').setLevel(logging.INFO)
```

5. ¿Por Qué SQLAlchemy?

SQLAlchemy es el ORM más maduro y poderoso de Python:

Característica	SQLAlchemy	Otros ORMs
Madurez	18+ años	Variable
Async nativo	✅ (2.0+)	Parcial
Type hints	✅ Completo	Parcial
Flexibilidad	Alta (Core + ORM)	Limitada
Comunidad	Muy grande	Variable
Documentación	Excelente	Variable

```
# SQLAlchemy 2.0 - Estilo moderno con type hints
from sqlalchemy.orm import Mapped, mapped_column

class User(Base):
    __tablename__ = "users"

    # Type hints integrados - IDE sabe el tipo
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
```

6. SQLAlchemy Core vs ORM

SQLAlchemy tiene dos capas:

Core (Bajo nivel)

```
from sqlalchemy import Table, Column, Integer, String, MetaData, insert

metadata = MetaData()

users = Table(
    "users",
    metadata,
    Column("id", Integer, primary_key=True),
    Column("name", String(100)),
)

# Operaciones con tablas directamente
stmt = insert(users).values(name="John")
connection.execute(stmt)
```

ORM (Alto nivel) - Lo que usaremos

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))

# Operaciones con objetos
user = User(name="John")
session.add(user)
```

💡 *En este bootcamp usamos el ORM porque es más intuitivo y productivo para aplicaciones web.*

7. Sync vs Async

SQLAlchemy 2.0 soporta ambos modos:

Síncrono (más simple)

```
from sqlalchemy import create_engine
from sqlalchemy.orm import Session

engine = create_engine("sqlite:///./app.db")

with Session(engine) as session:
    user = session.get(User, 1)
```

Asíncrono (mejor rendimiento)

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession

engine = create_async_engine("sqlite+aiosqlite:///./app.db")

async with AsyncSession(engine) as session:
    result = await session.get(User, 1)
```

💡 **Comenzaremos con sync** para entender los conceptos. Luego migraremos a async.

Verificación de Conceptos

Responde mentalmente:

1. ¿Qué problema resuelve un ORM?
2. ¿Cómo previene SQL injection un ORM?
3. ¿Cuál es la diferencia entre Core y ORM en SQLAlchemy?
4. ¿Por qué SQLAlchemy es popular en el ecosistema Python?

Recursos Adicionales

- [SQLAlchemy ORM Tutorial](#)
- [What's New in SQLAlchemy 2.0](#)
- [FastAPI SQL Databases](#)

Checklist

- ☐ Entiendo qué es un ORM
- ☐ Conozco las ventajas de usar SQLAlchemy
- ☐ Sé la diferencia entre Core y ORM
- ☐ Entiendo por qué comenzamos con sync

[Siguiente: Configuración de SQLAlchemy →](#)

Configuración de SQLAlchemy

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

- ☒ Crear un Engine de conexión a base de datos
- ☒ Configurar sesiones para operaciones
- ☒ Definir la Base declarativa para modelos
- ☒ Entender el flujo de conexión de SQLAlchemy

Contenido

 Arquitectura SQLAlchemy

1. Componentes Principales

SQLAlchemy tiene tres componentes clave:

Componente	Propósito	Analogía
Engine	Conexión a la base de datos	"La fábrica de conexiones"
Session	Unidad de trabajo (transacciones)	"El carrito de compras"
Base	Clase padre de todos los modelos	"El molde de galletas"

2. El Engine

El **Engine** es el punto de entrada a la base de datos. Gestiona el pool de conexiones.

```
from sqlalchemy import create_engine

# SQLite (archivo local)
engine = create_engine(
    "sqlite:///app.db",
    echo=True # Muestra SQL generado (útil para debug)
)

# SQLite en memoria (para tests)
engine = create_engine("sqlite:///memory:")

# PostgreSQL (producción)
engine = create_engine(
    "postgresql://user:password@localhost:5432/dbname",
    pool_size=5,          # Conexiones en el pool
    max_overflow=10,      # Conexiones extra permitidas
    pool_pre_ping=True    # Verifica conexiones antes de usar
)
```

URLs de Conexión

```
# Formato general
# dialect+driver://username:password@host:port/database

# SQLite
"sqlite:///app.db"          # Archivo relativo
"sqlite:///absolute/path.db" # Archivo absoluto
"sqlite:///memory:"        # En memoria

# PostgreSQL
"postgresql://user:pass@localhost:5432/mydb"
"postgresql+psycopg://user:pass@localhost:5432/mydb" # Con driver específico

# MySQL
"mysql+pymysql://user:pass@localhost:3306/mydb"
```

3. La Session

La **Session** es donde ocurre la magia. Gestiona las transacciones y el estado de los objetos.

```
from sqlalchemy.orm import Session, sessionmaker
```

```

# Opción 1: Crear session directamente
with Session(engine) as session:
    # Operaciones aquí
    session.commit()

# Opción 2: Crear una fábrica de sessions (recomendado)
SessionLocal = sessionmaker(
    bind=engine,
    autocommit=False, # Control manual de transacciones
    autoflush=False   # Control manual de flush
)

# Usar la fábrica
with SessionLocal() as session:
    # Operaciones aquí
    session.commit()

```

Ciclo de Vida de una Session

```

# 1. Crear session
session = SessionLocal()

try:
    # 2. Realizar operaciones
    user = User(name="John")
    session.add(user)

    # 3. Commit - persiste los cambios
    session.commit()

except Exception:
    # 4. Rollback en caso de error
    session.rollback()
    raise

finally:
    # 5. Cerrar session
    session.close()

# Forma pythónica con context manager
with SessionLocal() as session:
    user = User(name="John")
    session.add(user)
    session.commit() # Rollback automático si hay excepción

```

4. La Base Declarativa

La **Base** es la clase de la que heredan todos tus modelos:

```

from sqlalchemy.orm import DeclarativeBase

# SQLAlchemy 2.0 - Estilo moderno
class Base(DeclarativeBase):
    pass

```

```
# Todos los modelos heredan de Base
class User(Base):
    __tablename__ = "users"
    # ...

class Product(Base):
    __tablename__ = "products"
    # ...
```

Crear las Tablas

```
# Crea todas las tablas definidas en modelos que heredan de Base
Base.metadata.create_all(bind=engine)

# También puedes eliminarlas (¡cuidado en producción!)
Base.metadata.drop_all(bind=engine)
```

5. Estructura de Archivos Recomendada

```
src/
├─ database.py      # Configuración de SQLAlchemy
├─ models.py       # Modelos (tablas)
├─ schemas.py      # Schemas Pydantic
└─ main.py         # FastAPI app
```

database.py

```
"""
Database configuration for SQLAlchemy
"""

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

# URL de conexión (usar variables de entorno en producción)
SQLALCHEMY_DATABASE_URL = "sqlite:///./app.db"

# Crear engine
engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False} # Solo para SQLite
)

# Fábrica de sessions
SessionLocal = sessionmaker(
    bind=engine,
    autocommit=False,
    autoflush=False
)

# Base declarativa
class Base(DeclarativeBase):
    pass
```



```
def get_db():
    """
    Dependency que provee una session de base de datos.
    Se usa en FastAPI con Depends().
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

6. Configuración con Variables de Entorno

```
# database.py - versión con configuración externa
import os
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

# Leer de variable de entorno con fallback
DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "sqlite:///./app.db" # Default para desarrollo
)

# Ajustes específicos por base de datos
connect_args = {}
if DATABASE_URL.startswith("sqlite"):
    connect_args["check_same_thread"] = False

engine = create_engine(
    DATABASE_URL,
    connect_args=connect_args,
    echo=os.getenv("SQL_ECHO", "false").lower() == "true"
)

SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

class Base(DeclarativeBase):
    pass
```

.env

```
# Desarrollo
DATABASE_URL=sqlite:///./app.db
SQL_ECHO=true

# Producción
DATABASE_URL=postgresql://user:pass@db:5432/myapp
SQL_ECHO=false
```

7. Verificar la Conexión

```
# test_connection.py
from sqlalchemy import text
from database import engine

def test_connection():
    """Verifica que la conexión funciona"""
    with engine.connect() as connection:
        result = connection.execute(text("SELECT 1"))
        print(f"Conexión exitosa: {result.scalar()}")

if __name__ == "__main__":
    test_connection()
```

Ejemplo Completo

```
# database.py
from sqlalchemy import create_engine, text
from sqlalchemy.orm import DeclarativeBase, sessionmaker

SQLALCHEMY_DATABASE_URL = "sqlite:///./example.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False},
    echo=True # Ver SQL en consola
)

SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# Verificar conexión
if __name__ == "__main__":
    with engine.connect() as conn:
        result = conn.execute(text("SELECT sqlite_version()"))
        print(f"SQLite version: {result.scalar()}")
    print("✅ Conexión configurada correctamente")
```

Checklist

- ☐ Entiendo qué es el Engine
- ☐ Sé crear y usar Sessions
- ☐ Puedo definir la Base declarativa
- ☐ Conozco la estructura de archivos recomendada

Modelos Declarativos

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

-  Definir modelos como clases Python
-  Usar tipos de datos SQLAlchemy correctamente
-  Configurar primary keys, defaults y constraints
-  Aplicar el nuevo estilo de SQLAlchemy 2.0

Contenido

1. Anatomía de un Modelo

Un modelo SQLAlchemy representa una tabla en la base de datos:

```
from datetime import datetime
from sqlalchemy import String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class User(Base):
    # Nombre de la tabla en la base de datos
    __tablename__ = "users"

    # Columnas con type hints (SQLAlchemy 2.0)
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True)
    age: Mapped[int | None] = mapped_column(default=None)
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

Componentes Clave

Componente	Propósito
<code>__tablename__</code>	Nombre de la tabla SQL
<code>Mapped[T]</code>	Declara el tipo Python del atributo
<code>mapped_column()</code>	Configura la columna SQL

2. Tipos de Datos

Mapeo Python → SQLAlchemy → SQL

Python Type	SQLAlchemy	SQL (SQLite)	SQL (PostgreSQL)
int	Integer	INTEGER	INTEGER
str	String(n)	VARCHAR(n)	VARCHAR(n)
str	Text	TEXT	TEXT
float	Float	REAL	DOUBLE PRECISION
bool	Boolean	INTEGER	BOOLEAN
datetime	DateTime	TIMESTAMP	TIMESTAMP
date	Date	DATE	DATE
bytes	LargeBinary	BLOB	BYTEA
Decimal	Numeric	NUMERIC	NUMERIC

```

from datetime import datetime, date
from decimal import Decimal
from sqlalchemy import String, Text, Numeric, LargeBinary
from sqlalchemy.orm import Mapped, mapped_column

class Product(Base):
    __tablename__ = "products"

    id: Mapped[int] = mapped_column(primary_key=True)

    # String con longitud máxima
    name: Mapped[str] = mapped_column(String(200))

    # Texto largo sin límite
    description: Mapped[str | None] = mapped_column(Text, default=None)

    # Decimal para precios (precisión importante)
    price: Mapped[Decimal] = mapped_column(Numeric(10, 2))

    # Entero
    stock: Mapped[int] = mapped_column(default=0)

    # Booleano
    is_available: Mapped[bool] = mapped_column(default=True)

    # Fechas
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
    release_date: Mapped[date | None] = mapped_column(default=None)

```

3. Tipos Opcionales (Nullable)

```

from sqlalchemy.orm import Mapped, mapped_column

class User(Base):
    __tablename__ = "users"

```

```

id: Mapped[int] = mapped_column(primary_key=True)

# Requerido (NOT NULL) - sin | None
email: Mapped[str] = mapped_column(String(255), unique=True)

# Opcional (NULL permitido) - con | None
phone: Mapped[str | None] = mapped_column(String(20), default=None)
bio: Mapped[str | None] = mapped_column(Text, default=None)

# También opcional
age: Mapped[int | None] = mapped_column(default=None)

```

El type hint determina si la columna es nullable:

- `Mapped[str]` → NOT NULL
- `Mapped[str | None]` → NULL permitido

4. Primary Keys

```

from sqlalchemy.orm import Mapped, mapped_column

class User(Base):
    __tablename__ = "users"

    # Autoincrement por defecto con integer primary key
    id: Mapped[int] = mapped_column(primary_key=True)

# Primary key con UUID
import uuid
from sqlalchemy import Uuid

class Session(Base):
    __tablename__ = "sessions"

    id: Mapped[uuid.UUID] = mapped_column(
        Uuid,
        primary_key=True,
        default=uuid.uuid4
    )

```

5. Valores por Defecto

```

from datetime import datetime
from sqlalchemy import func
from sqlalchemy.orm import Mapped, mapped_column

class Article(Base):
    __tablename__ = "articles"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))

```

```

# Default Python (se evalúa en Python)
is_published: Mapped[bool] = mapped_column(default=False)

# Default con función (se evalúa al insertar)
created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

# Default SQL (lo ejecuta la base de datos)
updated_at: Mapped[datetime] = mapped_column(
    server_default=func.now(), # SQL: DEFAULT CURRENT_TIMESTAMP
    onupdate=datetime.utcnow # Se actualiza en cada UPDATE
)

# Default con valor fijo
views: Mapped[int] = mapped_column(default=0)
status: Mapped[str] = mapped_column(String(20), default="draft")

```

default vs server_default

```

# default - Python genera el valor antes de INSERT
created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
# INSERT INTO articles (created_at, ...) VALUES ('2024-01-15 10:30:00', ...)

# server_default - La base de datos genera el valor
created_at: Mapped[datetime] = mapped_column(server_default=func.now())
# INSERT INTO articles (...) VALUES (...) -- DB usa CURRENT_TIMESTAMP

```

6. Constraints (Restricciones)

```

from sqlalchemy import String, CheckConstraint, UniqueConstraint
from sqlalchemy.orm import Mapped, mapped_column

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)

    # Unique - valor único en toda la tabla
    email: Mapped[str] = mapped_column(String(255), unique=True)

    # Index - mejora búsquedas
    username: Mapped[str] = mapped_column(String(50), unique=True, index=True)

    age: Mapped[int | None] = mapped_column(default=None)
    role: Mapped[str] = mapped_column(String(20), default="user")

    # Constraints a nivel de tabla
    __table_args__ = (
        # Check constraint - validación SQL
        CheckConstraint("age >= 0 AND age <= 150", name="valid_age"),
        CheckConstraint("role IN ('user', 'admin', 'moderator')", name="valid_role"),

        # Unique constraint compuesto

```

```
        UniqueConstraint("email", "role", name="unique_email_per_role"),
    )
```

7. Ejemplo Completo: Sistema de Blog

```
"""
models.py - Modelos para un sistema de blog
"""

from datetime import datetime
from sqlalchemy import String, Text, ForeignKey, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class Author(Base):
    """Modelo de autor"""
    __tablename__ = "authors"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    bio: Mapped[str | None] = mapped_column(Text, default=None)
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

    def __repr__(self) -> str:
        return f"Author(id={self.id}, name={self.name!r})"

class Post(Base):
    """Modelo de publicación"""
    __tablename__ = "posts"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    slug: Mapped[str] = mapped_column(String(250), unique=True, index=True)
    content: Mapped[str] = mapped_column(Text)
    summary: Mapped[str | None] = mapped_column(String(500), default=None)

    # Foreign key (lo veremos en detalle en la siguiente semana)
    author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))

    # Estados y metadata
    is_published: Mapped[bool] = mapped_column(default=False)
    views: Mapped[int] = mapped_column(default=0)

    # Timestamps
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
    updated_at: Mapped[datetime | None] = mapped_column(
        default=None,
        onupdate=datetime.utcnow
    )
```

```

published_at: Mapped[datetime | None] = mapped_column(default=None)

def __repr__(self) -> str:
    return f"Post(id={self.id}, title={self.title!r})"

class Tag(Base):
    """Modelo de etiqueta"""
    __tablename__ = "tags"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50), unique=True)
    slug: Mapped[str] = mapped_column(String(60), unique=True, index=True)

    def __repr__(self) -> str:
        return f"Tag(id={self.id}, name={self.name!r})"

```

8. Crear las Tablas

```

# main.py o script separado
from database import engine, Base
from models import Author, Post, Tag # Importar todos los modelos

# Crear todas las tablas
Base.metadata.create_all(bind=engine)
print("✅ Tablas creadas")

# Ver las tablas creadas
for table in Base.metadata.tables:
    print(f" - {table}")

```

Verificación

```

# Verificar que los modelos están bien definidos
from models import Author, Post, Tag
from database import engine, Base

# Esto muestra el SQL que se generaría
from sqlalchemy.schema import CreateTable

print(CreateTable(Author.__table__).compile(engine))

```

Checklist

- ☐ Sé definir modelos con `__tablename__`
- ☐ Uso `Mapped[T]` y `mapped_column()` correctamente
- ☐ Entiendo los tipos de datos y su mapeo
- ☐ Sé configurar defaults y constraints
- ☐ Puedo crear las tablas con `create_all()`



Operaciones CRUD



Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

- ☒ Crear registros en la base de datos
 - ☒ Leer registros con filtros y paginación
 - ☒ Actualizar registros existentes
 - ☒ Eliminar registros de forma segura
-



Contenido



Flujo CRUD

1. Setup Inicial

```
# database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

engine = create_engine("sqlite:///./crud_example.db", echo=True)
SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

class Base(DeclarativeBase):
    pass

# models.py
from datetime import datetime
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column
from database import Base

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    age: Mapped[int | None] = mapped_column(default=None)
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

# Crear tablas
Base.metadata.create_all(bind=engine)
```

2. CREATE - Crear Registros

Crear un solo registro

```

from sqlalchemy.orm import Session
from database import SessionLocal
from models import User

def create_user(name: str, email: str, age: int | None = None) -> User:
    """Crea un nuevo usuario"""
    with SessionLocal() as session:
        # 1. Crear instancia del modelo
        user = User(name=name, email=email, age=age)

        # 2. Agregar a la session
        session.add(user)

        # 3. Commit para persistir
        session.commit()

        # 4. Refresh para obtener valores generados (id, created_at)
        session.refresh(user)

    return user

# Uso
user = create_user("John Doe", "john@example.com", 25)
print(f"Usuario creado: {user.id} - {user.name}")

```

Crear múltiples registros

```

def create_users_bulk(users_data: list[dict]) -> list[User]:
    """Crea múltiples usuarios en una transacción"""
    with SessionLocal() as session:
        users = [User(**data) for data in users_data]

        # add_all para múltiples objetos
        session.add_all(users)
        session.commit()

        # Refresh cada uno
        for user in users:
            session.refresh(user)

    return users

# Uso
users_data = [
    {"name": "Alice", "email": "alice@example.com", "age": 30},
    {"name": "Bob", "email": "bob@example.com", "age": 28},
    {"name": "Charlie", "email": "charlie@example.com"},
]
users = create_users_bulk(users_data)

```

3. READ - Leer Registros

SQLAlchemy 2.0 usa `select()` para queries:

```
from sqlalchemy import select
from sqlalchemy.orm import Session
```

Obtener por ID

```
def get_user_by_id(user_id: int) -> User | None:
    """Obtiene un usuario por su ID"""
    with SessionLocal() as session:
        # Método directo para primary key
        user = session.get(User, user_id)
        return user

# Uso
user = get_user_by_id(1)
if user:
    print(f"Encontrado: {user.name}")
else:
    print("Usuario no encontrado")
```

Obtener por campo único

```
def get_user_by_email(email: str) -> User | None:
    """Obtiene un usuario por email"""
    with SessionLocal() as session:
        stmt = select(User).where(User.email == email)
        user = session.execute(stmt).scalar_one_or_none()
        return user

# Alternativas para scalar
# .scalar_one() - Exactamente uno, error si no o si hay más
# .scalar_one_or_none() - Uno o None, error si hay más
# .scalars().first() - El primero o None
```

Listar todos

```
def get_all_users() -> list[User]:
    """Obtiene todos los usuarios"""
    with SessionLocal() as session:
        stmt = select(User)
        result = session.execute(stmt)
        users = result.scalars().all()
        return users
```

Filtrar con condiciones

```
def get_active_users(min_age: int | None = None) -> list[User]:
    """Obtiene usuarios activos, opcionalmente filtrados por edad"""
    with SessionLocal() as session:
        stmt = select(User).where(User.is_active == True)

        if min_age is not None:
            stmt = stmt.where(User.age >= min_age)
```

```

        result = session.execute(stmt)
        return result.scalars().all()

# Operadores de comparación
# User.age == 25      # Igual
# User.age != 25      # Diferente
# User.age > 25       # Mayor que
# User.age >= 25      # Mayor o igual
# User.age < 25       # Menor que
# User.age <= 25      # Menor o igual
# User.age.is_(None)  # IS NULL
# User.age.is_not(None) # IS NOT NULL

```

Filtros avanzados

```

from sqlalchemy import select, or_, and_

def search_users(
    name: str | None = None,
    email: str | None = None,
    min_age: int | None = None,
    max_age: int | None = None
) -> list[User]:
    """Búsqueda avanzada de usuarios"""
    with SessionLocal() as session:
        stmt = select(User).where(User.is_active == True)

        # LIKE para búsqueda parcial
        if name:
            stmt = stmt.where(User.name.ilike(f"%{name}%"))

        if email:
            stmt = stmt.where(User.email.ilike(f"%{email}%"))

        # Rango de edad
        if min_age is not None:
            stmt = stmt.where(User.age >= min_age)
        if max_age is not None:
            stmt = stmt.where(User.age <= max_age)

        return session.execute(stmt).scalars().all()

# OR y AND explícitos
stmt = select(User).where(
    or_(
        User.name.ilike("%john%"),
        User.email.ilike("%john%")
    )
)

stmt = select(User).where(
    and_(
        User.is_active == True,
        User.age >= 18
    )
)

```

```
)  
)
```

Ordenar resultados

```
from sqlalchemy import select, desc, asc  
  
def get_users_sorted(order_by: str = "name", descending: bool = False) -> list[User]:  
    """Obtiene usuarios ordenados"""  
    with SessionLocal() as session:  
        stmt = select(User)  
  
        # Ordenar dinámicamente  
        column = getattr(User, order_by)  
        if descending:  
            stmt = stmt.order_by(desc(column))  
        else:  
            stmt = stmt.order_by(asc(column))  
  
        return session.execute(stmt).scalars().all()  
  
# Múltiples columnas de ordenamiento  
stmt = select(User).order_by(  
    desc(User.created_at),  
    asc(User.name)  
)
```

Paginación

```
def get_users_paginated(  
    skip: int = 0,  
    limit: int = 10  
) -> tuple[list[User], int]:  
    """Obtiene usuarios con paginación"""  
    with SessionLocal() as session:  
        # Query base  
        base_stmt = select(User).where(User.is_active == True)  
  
        # Contar total  
        from sqlalchemy import func  
        count_stmt = select(func.count()).select_from(base_stmt.subquery())  
        total = session.execute(count_stmt).scalar()  
  
        # Aplicar paginación  
        stmt = base_stmt.offset(skip).limit(limit)  
        users = session.execute(stmt).scalars().all()  
  
        return users, total  
  
# Uso  
users, total = get_users_paginated(skip=0, limit=10)  
print(f"Mostrando {len(users)} de {total} usuarios")
```

4. UPDATE - Actualizar Registros

Actualizar un registro

```
def update_user(user_id: int, **kwargs) -> User | None:
    """Actualiza un usuario existente"""
    with SessionLocal() as session:
        # 1. Obtener el usuario
        user = session.get(User, user_id)

        if not user:
            return None

        # 2. Actualizar atributos
        for key, value in kwargs.items():
            if hasattr(user, key):
                setattr(user, key, value)

        # 3. Commit
        session.commit()
        session.refresh(user)

        return user

# Uso
updated_user = update_user(1, name="John Updated", age=26)
```

Actualización masiva

```
from sqlalchemy import update

def deactivate_old_users(days_inactive: int = 365) -> int:
    """Desactiva usuarios inactivos"""
    from datetime import datetime, timedelta

    cutoff_date = datetime.utcnow() - timedelta(days=days_inactive)

    with SessionLocal() as session:
        stmt = (
            update(User)
            .where(User.created_at < cutoff_date)
            .where(User.is_active == True)
            .values(is_active=False)
        )

        result = session.execute(stmt)
        session.commit()

        return result.rowcount # Número de filas afectadas

# Uso
affected = deactivate_old_users(365)
print(f"Usuarios desactivados: {affected}")
```

5. DELETE - Eliminar Registros

Eliminar un registro

```
def delete_user(user_id: int) -> bool:
    """Elimina un usuario por ID"""
    with SessionLocal() as session:
        user = session.get(User, user_id)

        if not user:
            return False

        session.delete(user)
        session.commit()

    return True

# Uso
deleted = delete_user(1)
print(f"Usuario eliminado: {deleted}")
```

Soft Delete (Recomendado)

En producción, es mejor "desactivar" en lugar de eliminar:

```
def soft_delete_user(user_id: int) -> bool:
    """Soft delete - marca como inactivo sin eliminar"""
    with SessionLocal() as session:
        user = session.get(User, user_id)

        if not user:
            return False

        user.is_active = False
        # user.deleted_at = datetime.utcnow() # Si tienes este campo
        session.commit()

    return True
```

Eliminación masiva

```
from sqlalchemy import delete

def delete_inactive_users() -> int:
    """Elimina usuarios inactivos"""
    with SessionLocal() as session:
        stmt = delete(User).where(User.is_active == False)

        result = session.execute(stmt)
        session.commit()

    return result.rowcount
```

6. Manejo de Errores

```
from sqlalchemy.exc import IntegrityError, NoResultFound

def create_user_safe(name: str, email: str) -> User | None:
    """Crea usuario con manejo de errores"""
    with SessionLocal() as session:
        try:
            user = User(name=name, email=email)
            session.add(user)
            session.commit()
            session.refresh(user)
            return user

        except IntegrityError:
            # Email duplicado u otra violación de constraint
            session.rollback()
            print(f"Error: Email {email} ya existe")
            return None

def get_user_strict(email: str) -> User:
    """Obtiene usuario o lanza excepción"""
    with SessionLocal() as session:
        stmt = select(User).where(User.email == email)

        try:
            user = session.execute(stmt).scalar_one()
            return user
        except NoResultFound:
            raise ValueError(f"Usuario con email {email} no encontrado")
```

7. Resumen de Métodos

Operación	Método SQLAlchemy 2.0
Crear	<code>session.add()</code> , <code>session.add_all()</code>
Leer por ID	<code>session.get(Model, id)</code>
Leer con filtro	<code>session.execute(select(...).where(...))</code>
Actualizar	<code>setattr()</code> + <code>commit()</code> O <code>update().values()</code>
Eliminar	<code>session.delete()</code> O <code>delete().where()</code>
Contar	<code>select(func.count())</code>
Paginar	<code>.offset(skip).limit(limit)</code>

✓ Checklist

- ☐ Sé crear registros con `add()` y `commit()`
- ☐ Puedo leer registros con `select()` y filtros
- ☐ Entiendo la diferencia entre `scalar_one()` y `scalar_one_or_none()`
- ☐ Puedo actualizar registros de forma individual y masiva

- ☐ Sé eliminar registros y entiendo soft delete
- ☐ Manejo errores con try/except y rollback

[← Anterior: Modelos Declarativos](#) | [Siguiente: Integración con FastAPI](#) →

Integración con FastAPI

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

- ☒ Integrar SQLAlchemy con FastAPI usando dependencies
- ☒ Crear endpoints CRUD conectados a base de datos
- ☒ Usar Pydantic para validación y SQLAlchemy para persistencia
- ☒ Manejar errores de base de datos en endpoints

Contenido

1. Dependency Injection para Database

FastAPI usa **Dependency Injection** para proveer la sesión de base de datos:

```
# database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker, Session
from typing import Generator

SQLALCHEMY_DATABASE_URL = "sqlite:///./app.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False} # Solo para SQLite
)

SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

class Base(DeclarativeBase):
    pass

def get_db() -> Generator[Session, None, None]:
    """
    Dependency que provee una sesión de base de datos.

    - Crea una nueva sesión para cada request
    - Cierra la sesión al finalizar (incluso si hay error)
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

2. Estructura de Archivos

```
src/
├─ main.py           # FastAPI app y endpoints
├─ database.py       # Configuración SQLAlchemy
├─ models.py         # Modelos SQLAlchemy (tablas)
└─ schemas.py        # Schemas Pydantic (validación)
```

3. Modelos SQLAlchemy vs Schemas Pydantic

Importante: Usamos modelos diferentes para cada propósito:

Archivo	Propósito	Librería
models.py	Definir tablas en DB	SQLAlchemy
schemas.py	Validar request/response	Pydantic

```
# models.py - Tabla en base de datos
from datetime import datetime
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column
from database import Base

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    hashed_password: Mapped[str] = mapped_column(String(255)) # ¡No exponer!
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

```
# schemas.py - Validación de datos
from datetime import datetime
from pydantic import BaseModel, EmailStr, Field, ConfigDict

class UserBase(BaseModel):
    """Campos comunes"""
    name: str = Field(..., min_length=2, max_length=100)
    email: EmailStr

class UserCreate(UserBase):
    """Schema para crear usuario"""
    password: str = Field(..., min_length=8)

class UserUpdate(BaseModel):
    """Schema para actualizar usuario (todos opcionales)"""
    name: str | None = Field(None, min_length=2, max_length=100)
    email: EmailStr | None = None
```

```

is_active: bool | None = None

class UserResponse(UserBase):
    """Schema de respuesta (sin password)"""
    id: int
    is_active: bool
    created_at: datetime

    # Permite crear desde objeto SQLAlchemy
    model_config = ConfigDict(from_attributes=True)

class UserListResponse(BaseModel):
    """Schema para lista paginada"""
    users: list[UserResponse]
    total: int
    skip: int
    limit: int

```

4. CRUD Completo con FastAPI

```

# main.py
from fastapi import FastAPI, Depends, HTTPException, status, Query
from sqlalchemy import select, func
from sqlalchemy.orm import Session
from sqlalchemy.exc import IntegrityError

from database import engine, Base, get_db
from models import User
from schemas import UserCreate, UserUpdate, UserResponse, UserListResponse

# Crear tablas al iniciar
Base.metadata.create_all(bind=engine)

app = FastAPI(title="Users API", version="1.0.0")

# =====
# CREATE
# =====

@app.post(
    "/users",
    response_model=UserResponse,
    status_code=status.HTTP_201_CREATED,
    tags=["users"]
)
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    """Crea un nuevo usuario"""
    # Verificar si email ya existe
    stmt = select(User).where(User.email == user.email)
    existing = db.execute(stmt).scalar_one_or_none()

    if existing:

```

```

        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=f"Email {user.email} already registered"
        )

# Crear usuario (en producción: hashear password!)
db_user = User(
    name=user.name,
    email=user.email,
    hashed_password=f"hashed_{user.password}" # ⚠ Solo ejemplo
)

db.add(db_user)
db.commit()
db.refresh(db_user)

return db_user

# =====
# READ
# =====

@app.get(
    "/users",
    response_model=UserListResponse,
    tags=["users"]
)
def list_users(
    skip: int = Query(0, ge=0, description="Offset for pagination"),
    limit: int = Query(10, ge=1, le=100, description="Limit results"),
    is_active: bool | None = Query(None, description="Filter by active status"),
    db: Session = Depends(get_db)
):
    """Lista usuarios con paginación y filtros"""
    # Query base
    base_stmt = select(User)

    # Aplicar filtros
    if is_active is not None:
        base_stmt = base_stmt.where(User.is_active == is_active)

    # Contar total
    count_stmt = select(func.count()).select_from(base_stmt.subquery())
    total = db.execute(count_stmt).scalar()

    # Aplicar paginación
    stmt = base_stmt.offset(skip).limit(limit).order_by(User.id)
    users = db.execute(stmt).scalars().all()

    return UserListResponse(
        users=users,
        total=total,
        skip=skip,
        limit=limit
    )

```

```

@app.get(
    "/users/{user_id}",
    response_model=UserResponse,
    tags=["users"]
)
def get_user(user_id: int, db: Session = Depends(get_db)):
    """Obtiene un usuario por ID"""
    user = db.get(User, user_id)

    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"User with id {user_id} not found"
        )

    return user

# =====
# UPDATE
# =====

@app.put(
    "/users/{user_id}",
    response_model=UserResponse,
    tags=["users"]
)
def update_user(
    user_id: int,
    user_update: UserUpdate,
    db: Session = Depends(get_db)
):
    """Actualiza un usuario existente"""
    user = db.get(User, user_id)

    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"User with id {user_id} not found"
        )

    # Actualizar solo campos proporcionados
    update_data = user_update.model_dump(exclude_unset=True)

    # Verificar email único si se está actualizando
    if "email" in update_data:
        stmt = select(User).where(
            User.email == update_data["email"],
            User.id != user_id
        )
        existing = db.execute(stmt).scalar_one_or_none()
        if existing:
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,

```

```

        detail=f"Email {update_data['email']} already in use"
    )

    for key, value in update_data.items():
        setattr(user, key, value)

    db.commit()
    db.refresh(user)

    return user

# =====
# DELETE
# =====

@app.delete(
    "/users/{user_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    tags=["users"]
)
def delete_user(user_id: int, db: Session = Depends(get_db)):
    """Elimina un usuario"""
    user = db.get(User, user_id)

    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"User with id {user_id} not found"
        )

    db.delete(user)
    db.commit()

    return None

# =====
# HEALTH CHECK
# =====

@app.get("/health", tags=["health"])
def health_check(db: Session = Depends(get_db)):
    """Verifica conexión a base de datos"""
    from sqlalchemy import text

    try:
        db.execute(text("SELECT 1"))
        return {"status": "healthy", "database": "connected"}
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail=f"Database unavailable: {str(e)}"
        )

```

5. El Flujo Completo

Request HTTP



FastAPI Endpoint

- Recibe datos
- Valida con Pydantic (schema)



Depends(get_db)

- Crea Session de SQLAlchemy



Lógica de Negocio

- Usa Session para queries
- Trabaja con modelos SQLAlchemy



Response

- FastAPI convierte modelo → schema
- Serializa a JSON



Response HTTP

6. Patrón de Separación (Servicios)

Para proyectos más grandes, separa la lógica en servicios:

```
# services/user_service.py
from sqlalchemy import select
from sqlalchemy.orm import Session
from models import User
from schemas import UserCreate, UserUpdate

class UserService:
    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, user_id: int) -> User | None:
        return self.db.get(User, user_id)

    def get_by_email(self, email: str) -> User | None:
        stmt = select(User).where(User.email == email)
        return self.db.execute(stmt).scalar_one_or_none()
```

```

def create(self, user_data: UserCreate) -> User:
    user = User(
        name=user_data.name,
        email=user_data.email,
        hashed_password=f"hashed_{user_data.password}"
    )
    self.db.add(user)
    self.db.commit()
    self.db.refresh(user)
    return user

def update(self, user: User, user_data: UserUpdate) -> User:
    update_dict = user_data.model_dump(exclude_unset=True)
    for key, value in update_dict.items():
        setattr(user, key, value)
    self.db.commit()
    self.db.refresh(user)
    return user

def delete(self, user: User) -> None:
    self.db.delete(user)
    self.db.commit()

# En el endpoint
@app.post("/users", response_model=UserResponse)
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    service = UserService(db)

    if service.get_by_email(user.email):
        raise HTTPException(status_code=409, detail="Email already exists")

    return service.create(user)

```

7. Tips de Producción

Usar variables de entorno

```

# database.py
import os

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./app.db")

```

Logging de SQL

```

# Para debug
engine = create_engine(DATABASE_URL, echo=True)

# Para producción
engine = create_engine(DATABASE_URL, echo=False)

```

Manejo de errores global


```
from fastapi import Request
from fastapi.responses import JSONResponse
from sqlalchemy.exc import SQLAlchemyError

@app.exception_handler(SQLAlchemyError)
async def sqlalchemy_exception_handler(request: Request, exc: SQLAlchemyError):
    return JSONResponse(
        status_code=500,
        content={"detail": "Database error occurred"}
    )
```

✓ Checklist

- ☐ Entiendo cómo funciona `Depends(get_db)`
- ☐ Sé la diferencia entre modelos SQLAlchemy y schemas Pydantic
- ☐ Puedo crear endpoints CRUD completos
- ☐ Uso `model_config = ConfigDict(from_attributes=True)` en schemas de respuesta
- ☐ Manejo errores con `HTTPException`

[← Anterior: Operaciones CRUD](#) | [Volver a README →](#)

🔧 Ejercicio 01: Configuración SQLAlchemy

🎯 Objetivo

Aprender a configurar SQLAlchemy: crear Engine, Session y Base declarativa.

Duración estimada: 30 minutos

📋 Pasos

Paso 1: Crear el Engine

El **Engine** es la conexión a la base de datos:

```
from sqlalchemy import create_engine

# SQLite guarda datos en un archivo local
engine = create_engine("sqlite:///./test.db", echo=True)
```

- `sqlite:///./test.db` → archivo `test.db` en directorio actual
- `echo=True` → muestra SQL generado (útil para debug)

Abre `starter/main.py` y descomenta la sección del Paso 1.

Paso 2: Crear la Base Declarativa

La **Base** es la clase padre de todos los modelos:

```
from sqlalchemy.orm import DeclarativeBase
```

```
class Base(DeclarativeBase):
    pass
```

Descomenta la sección del Paso 2 en `starter/main.py` .

Paso 3: Crear un Modelo Simple

Un modelo representa una tabla:

```
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

class Item(Base):
    __tablename__ = "items"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
```

Descomenta la sección del Paso 3.

Paso 4: Crear las Tablas

SQLAlchemy crea las tablas con `create_all()` :

```
Base.metadata.create_all(bind=engine)
```

Descomenta la sección del Paso 4.

Paso 5: Configurar SessionLocal

El **sessionmaker** crea sesiones configuradas:

```
from sqlalchemy.orm import sessionmaker

SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)
```

Descomenta la sección del Paso 5.

Paso 6: Probar la Sesión

Usa la sesión para insertar y consultar:

```
with SessionLocal() as session:
    item = Item(name="Test Item")
    session.add(item)
    session.commit()
    print(f"Item creado con ID: {item.id}")
```

Descomenta la sección del Paso 6.

 **Ejecutar**

```
uv run python starter/main.py
```

Salida esperada:

```
=== Paso 1: Engine creado ===
=== Paso 2: Base declarativa lista ===
=== Paso 3: Modelo Item definido ===
=== Paso 4: Tablas creadas ===
=== Paso 5: SessionLocal configurado ===
=== Paso 6: Probando sesión ===
Item creado con ID: 1
Items en DB: [Item(id=1, name='Test Item')]
```

También verás un archivo `test.db` creado.

✓ Verificación

- ☐ El archivo `test.db` se creó
- ☐ El Engine muestra SQL en consola (`echo=True`)
- ☐ El Item se guardó con ID auto-generado
- ☐ Puedes consultar el Item guardado

🧠 Conceptos Clave

Componente	Propósito
Engine	Conexión a la base de datos
DeclarativeBase	Clase padre para modelos
sessionmaker	Factory de sesiones
Session	Unidad de trabajo para transacciones

[← Volver a Prácticas](#) | [Siguiente: Modelos Declarativos →](#)

📦 Ejercicio 02: Modelos Declarativos

🎯 Objetivo

Practicar la creación de modelos SQLAlchemy con diferentes tipos de datos, restricciones y valores por defecto.

Duración estimada: 40 minutos

📋 Pasos

Paso 1: Setup Inicial

Configuración base que usaremos en todo el ejercicio:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker
```

```
engine = create_engine("sqlite:///./models_test.db", echo=True)

class Base(DeclarativeBase):
    pass
```

Abre `starter/main.py` y descomenta la sección del Paso 1.

Paso 2: Modelo con Tipos Básicos

Tipos fundamentales en SQLAlchemy:

```
class Product(Base):
    __tablename__ = "products"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    price: Mapped[float]
    in_stock: Mapped[bool] = mapped_column(default=True)
```

- `Mapped[int]` → INTEGER en SQL
- `Mapped[str]` requiere `String(length)`
- `Mapped[float]` → FLOAT en SQL
- `Mapped[bool]` → BOOLEAN en SQL

Descomenta la sección del Paso 2.

Paso 3: Campos Opcionales (Nullable)

Usar `| None` para campos que pueden ser NULL:

```
class Customer(Base):
    __tablename__ = "customers"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str | None] = mapped_column(String(255)) # Nullable
    phone: Mapped[str | None] = mapped_column(String(20)) # Nullable
```

Descomenta la sección del Paso 3.

Paso 4: Valores por Defecto

Python defaults vs Server defaults:

```
from datetime import datetime

class Order(Base):
    __tablename__ = "orders"

    id: Mapped[int] = mapped_column(primary_key=True)
    status: Mapped[str] = mapped_column(String(20), default="pending")
    total: Mapped[float] = mapped_column(default=0.0)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

- `default=value` → Python genera el valor antes de INSERT
- `server_default="value"` → La DB genera el valor

Descomenta la sección del Paso 4.

Paso 5: Restricciones (Constraints)

Asegurar integridad de datos:

```
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(50), unique=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    age: Mapped[int | None] = mapped_column(CheckConstraint("age >= 0"))
```

- `unique=True` → No permite duplicados
- `index=True` → Crea índice para búsquedas rápidas
- `CheckConstraint` → Validación a nivel de DB

Descomenta la sección del Paso 5.

Paso 6: Crear Tablas y Probar

Crear todas las tablas y probar con datos:

```
Base.metadata.create_all(bind=engine)

with SessionLocal() as session:
    product = Product(name="Laptop", price=999.99)
    session.add(product)
    session.commit()
```

Descomenta la sección del Paso 6.

Ejecutar

```
uv run python starter/main.py
```

Salida esperada:

```
=== Paso 1: Setup completado ===
=== Paso 2: Modelo Product definido ===
=== Paso 3: Modelo Customer definido ===
=== Paso 4: Modelo Order definido ===
=== Paso 5: Modelo User definido ===
=== Paso 6: Probando modelos ===
Product: Product(id=1, name='Laptop', price=999.99, in_stock=True)
Customer: Customer(id=1, name='John', email=None, phone=None)
Order: Order(id=1, status='pending', total=0.0, created_at=...)
User: User(id=1, username='john_doe', email='john@example.com', age=25)
```

✓ Verificación

- ☐ Todos los modelos se crearon sin errores
- ☐ El archivo `models_test.db` existe
- ☐ Los defaults se aplican correctamente
- ☐ Los campos nullable aceptan None

🧠 Tabla de Tipos

Python	SQLAlchemy	SQL
int	Integer	INTEGER
str	String(n)	VARCHAR(n)
float	Float	FLOAT
bool	Boolean	BOOLEAN
datetime	DateTime	DATETIME
T None	nullable	NULL allowed

[← Anterior: Configuración](#) | [Siguiente: CRUD →](#)

↺ Ejercicio 03: Operaciones CRUD

🎯 Objetivo

Dominar las operaciones Create, Read, Update y Delete con SQLAlchemy 2.0.

Duración estimada: 45 minutos

📋 Pasos

Paso 1: Setup con Modelo Task

Modelo de tareas para practicar CRUD:

```
class Task(Base):
    __tablename__ = "tasks"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    completed: Mapped[bool] = mapped_column(default=False)
    priority: Mapped[int] = mapped_column(default=1)
```

Abre `starter/main.py` y descomenta la sección del Paso 1.

Paso 2: CREATE - Crear Registros

Insertar nuevos registros:

```
# Un registro
task = Task(title="Learn SQLAlchemy")
session.add(task)
session.commit()

# Múltiples registros
tasks = [Task(title="Task 1"), Task(title="Task 2")]
session.add_all(tasks)
session.commit()
```

Descomenta la sección del Paso 2.

Paso 3: READ - Obtener por ID

Usar `session.get()` para buscar por primary key:

```
task = session.get(Task, 1) # Busca task con id=1
if task:
    print(f"Encontrado: {task.title}")
```

Descomenta la sección del Paso 3.

Paso 4: READ - Listar con Filtros

Usar `select()` para queries complejas:

```
from sqlalchemy import select

# Todas las tareas
stmt = select(Task)
tasks = session.execute(stmt).scalars().all()

# Con filtro
stmt = select(Task).where(Task.completed == False)
pending = session.execute(stmt).scalars().all()
```

Descomenta la sección del Paso 4.

Paso 5: READ - Ordenar y Pagar

```
from sqlalchemy import select, desc

# Ordenar por prioridad descendente
stmt = select(Task).order_by(desc(Task.priority))

# Pagar: skip 0, limit 5
stmt = select(Task).offset(0).limit(5)
```

Descomenta la sección del Paso 5.

Paso 6: UPDATE - Modificar Registros

```
# Obtener y modificar
task = session.get(Task, 1)
task.completed = True
task.priority = 5
session.commit()
```

Descomenta la sección del Paso 6.

Paso 7: DELETE - Eliminar Registros

```
task = session.get(Task, 1)
session.delete(task)
session.commit()
```

Descomenta la sección del Paso 7.

Ejecutar

```
uv run python starter/main.py
```

Salida esperada:

```
=== Paso 1: Setup completado ===
=== Paso 2: CREATE ===
Creada: Task(id=1, title='Learn SQLAlchemy')
Creadas 3 tareas adicionales
=== Paso 3: READ por ID ===
Por ID: Task(id=1, title='Learn SQLAlchemy')
=== Paso 4: READ con filtros ===
Todas: 4 tareas
Pendientes: 4 tareas
Alta prioridad: 1 tareas
=== Paso 5: READ ordenar y paginar ===
Ordenadas por prioridad: [...]
Página 1 (2 items): [...]
=== Paso 6: UPDATE ===
Actualizada: Task(id=1, completed=True, priority=10)
=== Paso 7: DELETE ===
Eliminada tarea id=4
Tareas restantes: 3
```

Verificación

- ☐ CREATE: Puedo crear uno y múltiples registros
 - ☐ READ: Puedo buscar por ID con `session.get()`
 - ☐ READ: Puedo filtrar con `select().where()`
 - ☐ UPDATE: Puedo modificar atributos y hacer commit
 - ☐ DELETE: Puedo eliminar registros con `session.delete()`
-

Métodos Clave

Operación	Código SQLAlchemy 2.0
Create one	<code>session.add(obj)</code>
Create many	<code>session.add_all([...])</code>
Read by ID	<code>session.get(Model, id)</code>
Read all	<code>session.execute(select(Model)).scalars().all()</code>
Read filter	<code>select(Model).where(...)</code>
Update	<code>obj.attr = value + commit()</code>
Delete	<code>session.delete(obj) + commit()</code>

[← Anterior: Modelos](#) | [Siguiente: FastAPI + DB →](#)

Ejercicio 04: FastAPI + Database

Objetivo

Integrar SQLAlchemy con FastAPI usando Dependency Injection para crear una API CRUD completa.

Duración estimada: 45 minutos

Pasos

Paso 1: Estructura de Archivos

El ejercicio usa una estructura simple:

```
starter/  
├─ main.py      # Todo en un archivo para simplicidad  
└─ notes.db     # SQLite (se crea automáticamente)
```

Abre `starter/main.py` para comenzar.

Paso 2: Configuración de Base de Datos

Setup de Engine, Base y SessionLocal:

```
engine = create_engine("sqlite:///./notes.db")  
SessionLocal = sessionmaker(bind=engine)  
  
class Base(DeclarativeBase):  
    pass
```

Descomenta la sección del Paso 2.

Paso 3: Modelo SQLAlchemy

Modelo `Note` para la base de datos:

```

class Note(Base):
    __tablename__ = "notes"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(100))
    content: Mapped[str] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

```

Descomenta la sección del Paso 3.

Paso 4: Schemas Pydantic

Schemas para validación de request/response:

```

class NoteCreate(BaseModel):
    title: str = Field(..., min_length=1, max_length=100)
    content: str

class NoteResponse(BaseModel):
    id: int
    title: str
    content: str
    created_at: datetime

    model_config = ConfigDict(from_attributes=True)

```

- `NoteCreate` : Valida datos de entrada
- `NoteResponse` : Serializa datos de salida
- `from_attributes=True` : Permite crear desde objeto SQLAlchemy

Descomenta la sección del Paso 4.

Paso 5: Dependency get_db

La función que provee la sesión:

```

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

- `yield` : Permite que FastAPI use la sesión
- `finally` : Garantiza que se cierre aunque haya error

Descomenta la sección del Paso 5.

Paso 6: Endpoint CREATE

```

@app.post("/notes", response_model=NoteResponse, status_code=201)
def create_note(note: NoteCreate, db: Session = Depends(get_db)):
    db_note = Note(title=note.title, content=note.content)
    db.add(db_note)

```

```
db.commit()
db.refresh(db_note)
return db_note
```

Descomenta la sección del Paso 6.

Paso 7: Endpoints READ

```
@app.get("/notes", response_model=list[NoteResponse])
def list_notes(db: Session = Depends(get_db)):
    stmt = select(Note).order_by(desc(Note.created_at))
    return db.execute(stmt).scalars().all()

@app.get("/notes/{note_id}", response_model=NoteResponse)
def get_note(note_id: int, db: Session = Depends(get_db)):
    note = db.get(Note, note_id)
    if not note:
        raise HTTPException(status_code=404, detail="Note not found")
    return note
```

Descomenta la sección del Paso 7.

Paso 8: Endpoints UPDATE y DELETE

```
@app.put("/notes/{note_id}", response_model=NoteResponse)
def update_note(note_id: int, note_data: NoteUpdate, db: Session = Depends(get_db)):
    note = db.get(Note, note_id)
    if not note:
        raise HTTPException(status_code=404, detail="Note not found")
    # Actualizar campos
    for key, value in note_data.model_dump(exclude_unset=True).items():
        setattr(note, key, value)
    db.commit()
    return note

@app.delete("/notes/{note_id}", status_code=204)
def delete_note(note_id: int, db: Session = Depends(get_db)):
    note = db.get(Note, note_id)
    if not note:
        raise HTTPException(status_code=404, detail="Note not found")
    db.delete(note)
    db.commit()
```

Descomenta la sección del Paso 8.

Ejecutar

```
# Iniciar servidor
uv run fastapi dev starter/main.py
```

```
# 0 con uvicorn directamente
uv run uvicorn starter.main:app --reload
```

Abre <http://localhost:8000/docs> para ver Swagger UI.

Probar la API

Crear nota

```
curl -X POST http://localhost:8000/notes \
-H "Content-Type: application/json" \
-d '{"title": "Mi primera nota", "content": "Contenido de prueba"}'
```

Listar notas

```
curl http://localhost:8000/notes
```

Obtener una nota

```
curl http://localhost:8000/notes/1
```

Actualizar nota

```
curl -X PUT http://localhost:8000/notes/1 \
-H "Content-Type: application/json" \
-d '{"title": "Título actualizado"}'
```

Eliminar nota

```
curl -X DELETE http://localhost:8000/notes/1
```

Verificación

- ☐ El servidor inicia sin errores
- ☐ Swagger UI muestra los 5 endpoints
- ☐ POST /notes crea una nota y retorna con ID
- ☐ GET /notes lista todas las notas
- ☐ GET /notes/{id} retorna una nota o 404
- ☐ PUT /notes/{id} actualiza campos
- ☐ DELETE /notes/{id} elimina la nota

Flujo de Datos

```
Request → Pydantic (valida) → Endpoint → SQLAlchemy → DB
                                     ↓
Response ← Pydantic (serializa) ← Endpoint ← SQLAlchemy
```

Prácticas - Semana 05

Índice de Ejercicios

#	Ejercicio	Tema	Duración
01	Configuración SQLAlchemy	Engine, Session, Base	30 min
02	Modelos Declarativos	Mapped, mapped_column, tipos	40 min
03	Operaciones CRUD	Create, Read, Update, Delete	45 min
04	FastAPI + Database	Dependency Injection, endpoints	45 min

Tiempo total estimado: ~2.5 horas

Objetivo

Practicar los conceptos de SQLAlchemy ORM vistos en la teoría mediante ejercicios guiados paso a paso.

Formato de los Ejercicios

Cada ejercicio es un **tutorial guiado**:

1. Lee las instrucciones en el `README.md` del ejercicio
2. Abre el archivo `starter/main.py`
3. **Descomenta** el código según las instrucciones
4. Ejecuta y verifica que funciona

 **Importante:** El código ya está escrito y comentado. Tu tarea es descomentarlo, entenderlo y verificar que funciona correctamente.

Requisitos Previos

Antes de comenzar, asegúrate de tener:

```
# Crear entorno con uv
uv init week-05-practicas
cd week-05-practicas

# Agregar dependencias
uv add sqlalchemy fastapi uvicorn pydantic-settings

# Verificar instalación
uv run python -c "import sqlalchemy; print(f'SQLAlchemy {sqlalchemy.__version__}')"

```

Estructura de Cada Ejercicio

```
ejercicio-XX-nombre/
├── README.md          # Instrucciones paso a paso

```

```
└─ starter/
  └─ main.py      # Código con secciones comentadas
```

✓ Checklist de Completitud

- ☐ Ejercicio 01: Configuración SQLAlchemy
- ☐ Ejercicio 02: Modelos Declarativos
- ☐ Ejercicio 03: Operaciones CRUD
- ☐ Ejercicio 04: FastAPI + Database

[← Volver a Semana 05](#)

Proyecto Semana 05: API con SQLAlchemy ORM

Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio único]

 **IMPORTANTE:** Cada aprendiz trabaja sobre un dominio diferente.

Ejemplo Genérico de Referencia

Los ejemplos usan **"Warehouse"** (Almacén) que **NO** está en el pool. **Debes adaptar TODO a tu dominio asignado.**

Concepto	Ejemplo Genérico	Adapta a tu Dominio
Primary Entity	Item	{YourEntity}
Secondary Entity	Supplier	{YourRelated}
Relationship	N:1 (many items per supplier)	{your_relationship}

Objetivo

Construir una **API REST completa** usando SQLAlchemy ORM con SQLite, implementando relaciones 1:N entre dos entidades de tu dominio.

Requisitos Funcionales (Adapta a tu Dominio)

Entities (Mínimo 2 relacionadas)

Secondary Entity (el "uno" de la relación):

```
# Ejemplo genérico (Warehouse - Supplier)
class Supplier(Base):
    __tablename__ = "suppliers"

    id: Mapped[int] = mapped_column(primary_key=True)
    code: Mapped[str] = mapped_column(String(20), unique=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(100), unique=True)
    contact_name: Mapped[str | None]
    is_active: Mapped[bool] = mapped_column(default=True)
```

```
created_at: Mapped[datetime]

# Relationship
items: Mapped[list["Item"]] = relationship(back_populates="supplier")
```

Primary Entity (el "muchos" de la relación):

```
# Ejemplo genérico (Warehouse - Item)
class Item(Base):
    __tablename__ = "items"

    id: Mapped[int] = mapped_column(primary_key=True)
    sku: Mapped[str] = mapped_column(String(20), unique=True)
    name: Mapped[str] = mapped_column(String(200))
    supplier_id: Mapped[int] = mapped_column(ForeignKey("suppliers.id"))
    quantity: Mapped[int]
    status: Mapped[ItemStatus]
    created_at: Mapped[datetime]

# Relationship
supplier: Mapped["Supplier"] = relationship(back_populates="items")
```

Endpoints

Secondary Entity (Supplier):

Método	Endpoint	Descripción
POST	/suppliers/	Crear
GET	/suppliers/	Listar con paginación
GET	/suppliers/{id}	Obtener con sus items
PATCH	/suppliers/{id}	Actualizar
DELETE	/suppliers/{id}	Eliminar (si no tiene items)

Primary Entity (Item):

Método	Endpoint	Descripción
POST	/items/	Crear (validar FK)
GET	/items/	Listar (con filtro por supplier)
GET	/items/{id}	Obtener con supplier
PATCH	/items/{id}	Actualizar
DELETE	/items/{id}	Eliminar

 Estructura del Proyecto

```
starter/
├─ main.py
├─ database.py
```

```
├── models/
│   ├── __init__.py
│   ├── supplier.py
│   └── item.py
├── schemas/
│   ├── __init__.py
│   ├── supplier.py
│   └── item.py
├── routers/
│   ├── __init__.py
│   ├── suppliers.py
│   └── items.py
├── alembic/
├── pyproject.toml
├── Dockerfile
└── docker-compose.yml
```

✓ Criterios de Evaluación

Criterio	Puntos
Funcionalidad (40%)	
CRUD ambas entidades	15
Relación 1:N funciona	15
Alembic migraciones	10
Adaptación al Dominio (35%)	
Entidades coherentes con negocio	12
Relación lógica	13
Originalidad (no copia ejemplo)	10
Calidad del Código (25%)	
Models SQLAlchemy correctos	10
Eager/Lazy loading apropiado	10
Código limpio	5
Total	100

⚠ Política Anticopia

- ✗ **No copies** el ejemplo genérico "Supplier/Item"
- ✓ **Diseña** entidades específicas de tu dominio
- ✓ **Crea** relaciones lógicas para tu negocio

📖 Recursos

- [SQLAlchemy 2.0](#)
- [FastAPI + SQLAlchemy](#)

- [Pool de Dominios](#)

Tiempo estimado: 3 horas

[← Volver a Prácticas](#) | [Recursos →](#)



Glosario - Semana 05

A

Alembic

Herramienta de migración de bases de datos para SQLAlchemy. Permite versionar cambios en el esquema de la base de datos.

B

Base (DeclarativeBase)

Clase base de SQLAlchemy de la que heredan todos los modelos. Define el registro de metadatos para la creación de tablas.

```
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass
```

C

Commit

Operación que guarda permanentemente los cambios pendientes en la base de datos.

```
session.add(user)
session.commit() # Guarda el usuario en la DB
```

Connection Pool

Conjunto de conexiones de base de datos reutilizables que mejora el rendimiento evitando crear conexiones nuevas constantemente.

CRUD

Acrónimo de Create, Read, Update, Delete. Las cuatro operaciones básicas de persistencia de datos.

D

DeclarativeBase

Nueva clase base en SQLAlchemy 2.0 para definir modelos usando el estilo declarativo con type hints.

Dependency Injection

Patrón de diseño donde las dependencias (como la sesión de DB) se "inyectan" en las funciones que las necesitan.

```
def get_user(db: Session = Depends(get_db)):
    # db se inyecta automáticamente
```

E

Engine

Objeto central de SQLAlchemy que gestiona la conexión a la base de datos y el pool de conexiones.

```
engine = create_engine("sqlite:///./app.db")
```

F

Foreign Key

Restricción que crea una relación entre dos tablas, referenciando la clave primaria de otra tabla.

```
author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))
```

Flush

Operación que sincroniza los cambios en memoria con la base de datos, pero sin hacer commit (sin confirmar permanentemente).

I

Index

Estructura de datos que mejora la velocidad de las consultas en una columna específica.

```
email: Mapped[str] = mapped_column(index=True)
```

M

Mapped

Tipo genérico de SQLAlchemy 2.0 que indica el tipo Python de una columna.

```
name: Mapped[str] # Columna de tipo string
age: Mapped[int | None] # Columna nullable
```

mapped_column

Función que configura una columna de SQLAlchemy con opciones adicionales.

```
id: Mapped[int] = mapped_column(primary_key=True)
```

Metadata

Colección de objetos Table que representa la estructura de la base de datos.

```
Base.metadata.create_all(bind=engine)
```

N

Nullable

Columna que permite valores NULL. En SQLAlchemy 2.0 se indica con `T | None`.

```
phone: Mapped[str | None] # Puede ser NULL
```

O

ORM (Object-Relational Mapping)

Técnica que permite trabajar con bases de datos usando objetos Python en lugar de SQL directo.

P

Primary Key

Columna que identifica únicamente cada fila de una tabla.

```
id: Mapped[int] = mapped_column(primary_key=True)
```

Q

Query

Consulta a la base de datos. En SQLAlchemy 2.0 se usa `select()` en lugar del antiguo `query()`.

```
stmt = select(User).where(User.age > 18)
result = session.execute(stmt)
```

R

Refresh

Operación que recarga un objeto desde la base de datos para obtener valores generados (como IDs autoincrementales).

```
session.refresh(user) # Actualiza user.id después del commit
```

Rollback

Operación que deshace todos los cambios pendientes desde el último commit.

```
try:
    session.add(user)
    session.commit()
```

```
except:
    session.rollback()
```

S

Scalar

Método que retorna un solo valor de una consulta (en lugar de una fila completa).

```
user = session.execute(stmt).scalar_one_or_none()
```

Session

Objeto que gestiona la conversación con la base de datos. Maneja transacciones y el ciclo de vida de los objetos.

sessionmaker

Factory que crea sesiones con configuración predefinida.

```
SessionLocal = sessionmaker(bind=engine, autocommit=False)
```

T

Transaction

Conjunto de operaciones que se ejecutan como una unidad atómica. O todas tienen éxito, o ninguna.

U

Unique Constraint

Restricción que garantiza que no haya valores duplicados en una columna.

```
email: Mapped[str] = mapped_column(unique=True)
```

Unit of Work

Patrón de diseño implementado por Session que agrupa operaciones de base de datos.

Y

Yield

Palabra clave de Python usada en `get_db()` para crear un generador que provee la sesión y garantiza su cierre.

```
def get_db():
    db = SessionLocal()
    try:
        yield db # Provee la sesión
    finally:
        db.close() # Siempre se ejecuta
```



Semana 05: SQLAlchemy ORM - Introducción



Objetivos de la Semana

Al finalizar esta semana, serás capaz de:

- ✓ Entender qué es un ORM y sus beneficios
- ✓ Configurar SQLAlchemy 2.x con FastAPI
- ✓ Definir modelos de datos como clases Python
- ✓ Realizar operaciones CRUD con SQLAlchemy
- ✓ Integrar bases de datos en endpoints de FastAPI



Contenido

1. Teoría

Archivo	Tema	Duración
01-introduccion-orm.md	¿Qué es un ORM? SQLAlchemy vs SQL raw	20 min
02-configuracion-sqlalchemy.md	Engine, Session, Base declarativa	25 min
03-modelos-declarativos.md	Definir tablas como clases Python	25 min
04-operaciones-crud.md	Create, Read, Update, Delete	30 min
05-integracion-fastapi.md	Dependency injection y sessions	25 min

2. Prácticas

Ejercicio	Tema	Duración
ejercicio-01	Configurar SQLAlchemy con SQLite	30 min
ejercicio-02	Crear modelos declarativos	35 min
ejercicio-03	Operaciones CRUD básicas	40 min
ejercicio-04	Integrar con FastAPI	35 min

3. Proyecto

Proyecto	Descripción	Duración
Library API	API de biblioteca con libros y autores	90 min



Estructura de la Semana

```
week-05/  
├── README.md  
├── rubrica-evaluacion.md  
├── 0-assets/  
│   └── 01-orm-concept.svg
```

```
| | 02-sqlalchemy-architecture.svg
| | 03-crud-flow.svg
| 1-teoria/
| | 01-introduccion-orm.md
| | 02-configuracion-sqlalchemy.md
| | 03-modelos-declarativos.md
| | 04-operaciones-crud.md
| | 05-integracion-fastapi.md
| 2-practicas/
| | README.md
| | 01-ejercicio-configuracion/
| | 02-ejercicio-modelos/
| | 03-ejercicio-crud/
| | 04-ejercicio-fastapi-db/
| 3-proyecto/
| | README.md
| | starter/
| | solution/
| 4-recursos/
| | README.md
| | videografia/
| | webgrafia/
| 5-glosario/
| | README.md
```

Distribución del Tiempo

Actividad	Tiempo	Porcentaje
Teoría	2 horas	33%
Prácticas	2.5 horas	42%
Proyecto	1.5 horas	25%
Total	6 horas	100%

Requisitos Previos

Antes de comenzar esta semana, debes:

- ✔ Completar Semana 04 (Responses y Manejo de Errores)
- ✔ Entender async/await en Python
- ✔ Conocer Pydantic para validación
- ✔ Saber crear endpoints en FastAPI

Herramientas de la Semana

Herramienta	Versión	Propósito
SQLAlchemy	2.0+	ORM principal
SQLite	3.47+	Base de datos de desarrollo
aiosqlite	0.20+	Driver async para SQLite

Entregable

Proyecto: [Library API](#)

API de biblioteca funcionando con:

- ☐ CRUD de libros y autores
- ☐ SQLAlchemy ORM configurado
- ☐ Base de datos SQLite
- ☐ Documentación OpenAPI completa

Navegación

← Anterior	Actual	Siguiente →
Semana 04: Responses y Errores	Semana 05	Semana 06: Relaciones

Tip de la Semana

SQLAlchemy 2.0 introdujo un nuevo estilo "2.0" más pythónico y con mejor soporte para async. Este bootcamp usa exclusivamente el estilo moderno.

```
#  Estilo SQLAlchemy 2.0 (moderno)
from sqlalchemy import select
stmt = select(User).where(User.email == email)
result = session.execute(stmt)
user = result.scalar_one_or_none()

#  Estilo legacy 1.x (NO usar)
user = session.query(User).filter_by(email=email).first()
```

Recursos Rápidos

- [SQLAlchemy 2.0 Docs](#)
- [FastAPI SQL Databases](#)
- [Pydantic + SQLAlchemy](#)

Rúbrica de Evaluación - Semana 05

SQLAlchemy ORM - Introducción

Competencias a Evaluar

Competencia	Descripción
C1	Configurar SQLAlchemy con FastAPI
C2	Definir modelos declarativos
C3	Implementar operaciones CRUD

C4	Integrar base de datos con endpoints
----	--------------------------------------

Evidencias de Evaluación

1. Conocimiento (30%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
Conceptos ORM	Explica ventajas/desventajas de ORM vs SQL raw con ejemplos	Explica conceptos correctamente	Entiende conceptos básicos	No comprende el propósito de un ORM
Arquitectura SQLAlchemy	Describe Engine, Session, Base y su interacción	Conoce los componentes principales	Identifica algunos componentes	Confunde componentes
Tipos de datos	Conoce tipos SQLAlchemy y su mapeo a Python	Usa tipos correctamente	Usa tipos básicos	Errores en tipos de datos

2. Desempeño (40%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
Configuración	Configura conexión, session y modelos sin errores	Configuración funcional con pequeños ajustes	Requiere ayuda para configurar	No logra configurar SQLAlchemy
Modelos	Define modelos con tipos correctos, constraints y defaults	Modelos funcionales con campos básicos	Modelos incompletos pero funcionales	Errores en definición de modelos
CRUD	Implementa las 4 operaciones correctamente con manejo de errores	CRUD funcional, errores menores	Algunas operaciones funcionan	CRUD incompleto o con errores
Integración	Integra perfectamente con FastAPI usando dependencies	Integración funcional	Integración básica	No logra integrar

3. Producto (30%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
Library API	API completa con CRUD de libros y autores, documentada	API funcional con la mayoría de features	API parcialmente funcional	API no funcional
Código	Limpio, tipado, bien organizado en módulos	Código funcional y legible	Código funcional pero desorganizado	Código difícil de entender
Docker	Funciona con docker compose up sin errores	Funciona con ajustes menores	Requiere configuración manual	No funciona con Docker

Escala de Calificación

Nivel	Rango	Descripción
Excelente	90-100%	Dominio completo, puede explicar y enseñar
Bueno	80-89%	Comprensión sólida, implementa sin problemas
Suficiente	70-79%	Cumple requisitos mínimos
Insuficiente	<70%	No alcanza los objetivos mínimos



Checklist de Entrega

Ejercicios Prácticos

- ☐ Ejercicio 01: Configuración SQLAlchemy funcional
- ☐ Ejercicio 02: Modelos declarativos creados
- ☐ Ejercicio 03: Operaciones CRUD funcionando
- ☐ Ejercicio 04: Integración con FastAPI completa

Proyecto Library API

- ☐ CRUD de libros (Create, Read, Update, Delete)
- ☐ CRUD de autores
- ☐ Base de datos SQLite funcional
- ☐ Documentación OpenAPI generada
- ☐ Docker compose funcional
- ☐ Código organizado en módulos

Documentación

- ☐ Código comentado donde sea necesario
- ☐ README del proyecto actualizado
- ☐ Tipos de datos correctos (type hints)



Criterios Mínimos de Aprobación

Para aprobar esta semana, el estudiante debe:

- ☒ Obtener mínimo **70%** en cada tipo de evidencia
- ☒ Completar todos los ejercicios prácticos
- ☒ Entregar el proyecto Library API funcional
- ☒ El proyecto debe ejecutarse con `docker compose up`



Retroalimentación

Aspecto	Comentarios
Fortalezas	
Áreas de mejora	
Recomendaciones	

Calificación Final: ____ / 100

Estado: ☐ Aprobado | ☐ No Aprobado

Fecha de Evaluación: _____

Evaluador: _____
